

Programozás alapjai 3

Tóth Johanna Janka – G7FZYT

Virágültetés (logikai játék)

Házi Feladat Dokumentáció

1. Játékszabályok és eredményszámítás:

A játékot lehet játszani egyedül, vagy akár ketten is.

A pályák négyzetekből állnak, amik két típusra bonthatók: fal és út. Az egyes játékos a fel, le, jobbra, balra gombokkal tud lépni egy szomszédos (a két négyzetnek van egy-egy oldala, ami megegyezik koordinátáit tekintve) út típusú négyzetre, míg a kettes játékos a w, a, s, d gombokkal tudja ugyanezt csinálni.

Az nyeri meg a játékot, aki a játék elején kapott 10 virágját, gyorsabban és helyesen ülteti el. A pontszámítás az alábbi módon történik: $(20000 * ((\text{jóhelyen lévő virágok száma} * 100) - (\text{rossz helyre rakott virágok száma} * 50))) / (\text{játszott idő})$. Ezen felül a legkisebb kapható pont a nulla. (Negatív pont nem elérhető.)

A virágokat úgy szabályos elültetni (első játékos space-szel, második játékos shift-tel) a pályán, hogy az pontosan 3 fal típusú négyzettel érintkezhet (ha a sarkuk összeér a négyzeteknek, az is számít). A szabályos ültetésre jár a pont, szabálytalan ültetésnél pontlevonás jár.

A játék nem jelez, ha rossz helyre van ültetve a virág. A két játékos különböző virágokat ültet, mindenki a sajátját tudja csak lerakni és felvenni (ugyanúgy az ültetés gombbal). Az útra, ahova virág lett ültetve rá lehet lépni. Egy helyre csak egy virágot lehet ültetni.

A játéknak akkor van vége, ha az egyik (vagy egyetlen) játékosnak nincs több elültetendő virága.

2. Use-case-ek:

Amint elindul a program a játékos megadhatja, hogy az egy, vagy a kétszemélyes módot választja (, ez gombokkal lesz megvalósítva).

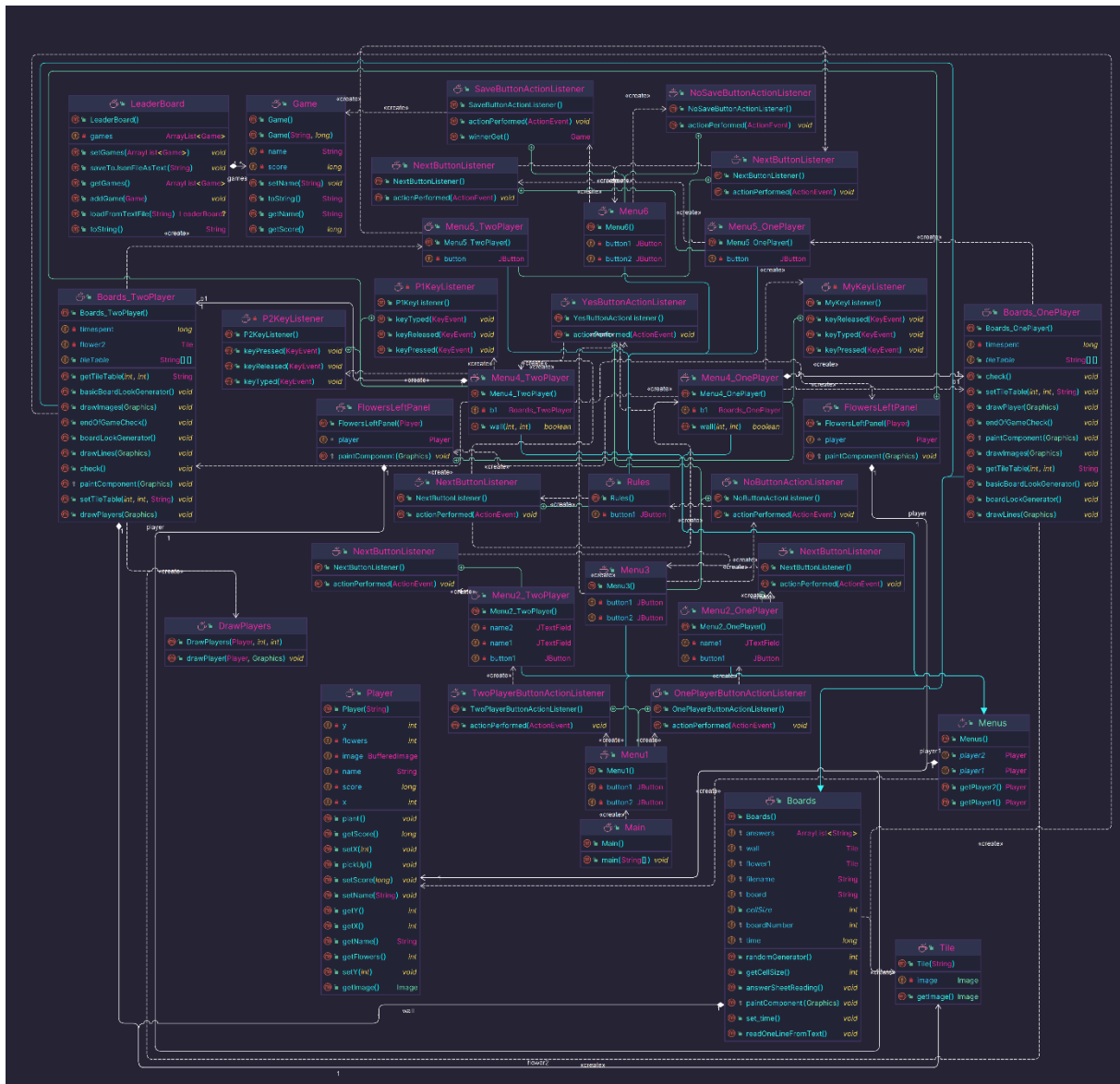
Majd megadhatja a játékos a nevét (két személyes mód esetén mindketten megadják egymás után).

Ezután a program megkérdezi a felhasználó(ka)t, hogy ismeri(k)-e a szabályzatot (, ez szintén gombokkal lesz megvalósítva). Ha nem ismeri, akkor elmagyarázza a program a szabályzatot.

A fenti bevezetők után kezdődik el igazából a játék és játszódik végig. A játék végén láthatod a pontszámokat, valamint a két személyes módban azt is, hogy melyik játékos nyert.

Ha a játékos szeretné, akkor le lehet menteni a „játszmát” a leaderboard.txt file-ba (JSON-nel menti le a Game objectet), ahol a score-ok szerint csökkenő sorrendben megtalálható lesz ez a játszma is a többi (korábban lementett) között. (Két személyes játék esetén csak a nyertes eredményét és nevét menti le a program. Döntetlen esetén a két játékos nevét egy „&” jellel elválasztva menti el névként és a közösen elért score-t.)

3. Osztálydiagram



4. Megoldások (package-ek, class-ok):

- Main.java:
 - Ez a fő program.
 - A 'public static void main(String[] args)' függvény meghívja az első menüt. (Ezen kívül nincsen más függvénye.)
- package player:
 - Player.java: a játékosok adatait tároló osztály
 - private int x,y ~ a játékos koordinátái a pályán
 - van gettere (public int) és settere (public void) is ugyanebben a classban (getX(), getY(), setX(int), setY(int))
 - private long score ~ a játékos által elért pontszám
 - van gettere (public long) és settere (public void) is (getScore(), setScore(long))
 - private String name ~ a játékos nevét tárolja
 - van van gettere (public String) és settere (public void) is (getName(), setName())
 - private int flowers ~ egy játékos a játék elején 10 virágot kap, a nem elültetett virágok aktuális számát tárolja
 - van gettere (public int) (getFlowers())
 - private BufferedImage image ~ ez a kép a játékos játéktáblán való megjelenési formáját tárolja (én design-oltam a karaktereket is)
 - van gettere (public Image) (getImage())
 - public Player(String imagePath) ~ a játékos konstruktora, csak úgy hozható létre játékos, ha a kinézetének a képét megadjuk paraméterként.
 - megnézi, hogy létezik-e a kép és ha igen, akkor beállítja az image-t
 - beállítja a flowers-t 10-re
 - public void plant() ~ levon egyet a virágok számából (0-ig megy)
 - public void pickUp() ~ hozzáad egyet a virágok számához (maximum 10-ig lehet menni)
 - Game.java: a játszmákat tároló osztály, (az elmenteni való class-om)
 - private String name ~ a győztes nevét tárolja
 - van gettere (public String) és settere (public void) is (getName(), setName())
 - private long score ~ a győztes eredményét tárolja
 - van gettere (public long) (getScore())
 - public Game() ~ üres konstruktor, kell a paraméter nélküli konstruktor, hogy JSON-nal le lehessen menteni
 - public Game(String, long) ~ két paraméteres konstruktor
 - beállítja a játszmát elmenteni kívánó játékos nevét, és az elért pontját (name, score)
 - public String toString()
 - LeaderBoard.java: ez az osztály végzi a fájlba írást, fájlból olvasást
 - private ArrayList<Game> games ~ a lementett játszmákat olvassa ide be a program és ehhez ad hozzá elemeket és ezt menti le a txt fájlba

- van gettere (public ArrayList<Game>) és settere (public void) is (getGames(), setGames(ArrayList<Game>))
 - public LeaderBoard() ~paraméter nélküli konstruktor, létrehozza a games listát
 - public void addGame(Game) ~ hozzáadja a games listához az új játékot és csökkenő sorba rakja a lista elemeit a score alapján (ranglistánál felül a legmasabb score-ral rendelkező áll)
 - public void saveToJsonAsText(String) ~ a stringgel (paraméter) megadott nevű fájlba lementi a JSON fájlt egy .txt kiterjesztésű fájlba
 - public static LeaderBoard loadFromTextFile(String) ~ a stringgel (paraméter) megadott fájlból (.txt kiterjesztésű) beolvassa az objektumot.
 - visszaad egy LeaderBoard-ot, ha nem létezett korábban a fájl akkor egy új LeaderBoard-ot készít (üreset)
 - public String toString()
- package menu:
 - Menus.java: Az összes menü ebből az absztrakt osztályból származik. Ez a class a JFrame leszármazottja
 - public static Player player1 ~ Itt hozza létre a program a játékosokat, hogy minden menüben el lehessen őket érni közvetlenül
 - van gettere (Player) (getPlayer())
 - public static Player player2 ~ Itt hozza létre a program a játékosokat, hogy minden menüben el lehessen őket érni közvetlenül
 - van gettere (Player) (getPlayer())
 - public Menus()
 - a Menük konstruktora, itt állítódik be, hogy ne lehessen a JFrame méretét állítani
 - Itt állítódik be, hogy a JFrame címe a „Planting Flowers” legyen, ami a játék címe
 - Itt kapnak a játékosok alapkoordinátákat (, ahol a játék kezdeténél állnak)
 - public Player getPlayer1() ~ visszaadja a player1 játékost
 - public Player getPlayer2() ~ visszaadja a player2 játékost
 - Menu1.java: az elsőnek megjelenő menü, ahol a játékos/játékosok kiválaszthatja/kiválaszthatják, hogy egy, vagy kétszemélyes módban szeretnének-e játszani
 - private JButton button1 ~ Egy játékos mód kiválasztásának a gombja
 - private JButton button2 ~ Két játékos mód kiválasztásának a gombja
 - public Menu1() ~ beállítja, hogy hogy nézzen ki a menü (Felül szöveg, alul két gomb, amik kattintásra aktiválódnak)
 - public class TwoPlayerButtonActionListener ~ meghív egy Menu2_TwoPlayer típusú menüt, majd ezt bezárja
 - public class OnePlayerButtonActionListener ~ meghív egy Menu2_OnePlayer típusú menüt, majd ezt bezárja
 - Menu2_OnePlayer.java:
 - private JTextField name1 ~ ebbe a TextField-be írhatja be a játékos a nevét, amit elment a player1-ba

- `private JButton button1` ~ ezt a gombot kell megnyomni, hogy a következő menübe léphessünk
- `public Menu2_OnePlayer()` ~ beállítja, hogy hogy nézzen ki a menü (Felül szöveg, alatta az írható felület, alul egy gomb)
- `public class NextButtonListener()` ~ kezeli, hogy a játékos mindenképpen adjon meg nevet, majd ha ez megtörtént, akkor beállítja a `player1` nevét a `TextField`-ben megadott szövegre, és tovább viszi a játékost a következő menüre (`Menu3`)
- `Menu2_TwoPlayer.java`: Ha a játékos két személyes módban szeretne játszani, akkor ez a menü nyílik meg neki másodíknak
 - `private JTextField name1` ~ ebbe a `TextField`-be írhatja be a játékos a nevét, amit elment a `player1`-ba
 - `private JTextField name2` ~ ebbe a `TextField`-be írhatja be a játékos a nevét, amit elment a `player2`-ba
 - `private JButton button1` ~ ezt a gombot kell megnyomni, hogy a következő menübe léphessünk
 - `public Menu2_TwoPlayer()` ~ beállítja, hogy hogy nézzen ki a menü (Felül szöveg, alatta az írható felület, alatta még egy írható felület, és legalul egy gomb)
 - `public class NextButtonListener()` ~ kezeli, hogy a játékosok mindenképpen adjanak meg nevet, majd ha ez megtörtént, akkor beállítja a `player1` és `player2` nevét a `TextField`-ekben megadott szövegekre, és tovább viszi a játékosokat a következő menüre (`Menu3`)
- `Menu3.java`: Itt lehet kérni a szabályok elmagyarázását a programtól, ez azonban opcionális lehet, hiszen lehet, hogy valaki már játszott korábban.
 - `private JButton button1` ~ elindítja magát a játékot, a játékos ennek a megnyomásával igazolja, hogy ismeri a játékot és tudja a szabályokat
 - `private JButton button2` ~ megnyitja a `Rules` (szabályok) menüt, mivel a játékos azt nyilatkozta ezzel a gombbal, hogy nem ismeri a játékot, és nem tudja a szabályokat
 - `public Menu3()` ~ beállítja, hogy hogy nézzen ki a menü (Felül egy szöveg, alul 2 gomb)
 - `public class YesButtonListener()` ~ elindítja a játékot, attól függetlenül, hogy egy, vagy kétjátékos módot választottunk korábban
 - `public class NoButtonActionListener()` ~ Megnyitja a `Rules` menüt, ahol a szabályokat ismerhetjük meg
- `Rules.java`: A szabályokat ismertető menü
 - `private JButton button1` ~ a továbblépés, játékot elindító gomb
 - `public Rules()` ~ beállítja a menü kinézetét (egy kép, ami elmagyarázza a szabályokat)
 - `public class NextButtonListener` ~ Ugyan azt csinálja, mint a `Menu3` `YesButtonListener()`-e
- `Menu4_OnePlayer.java`: Az egyjátékos játékot megjelenítő menü
 - `private Boards_OnePlayer b1` ~egy `JPanel` a `JFrame`-en belül, ami tulajdonképpen maga a játéklemező(amin belül a játékosok tudnak mozogni és virágot ültetni, és felvenni)

- `public Menu4_OnePlayer()` ~ beállítja, hogy hogy nézzen ki a menü (felül kiírja a játékos nevét, hogy hány virágja van jelenleg, alul pedig a játék tábla látható)
- `public class MyKeyListener()` ~ a fel, le, jobb, bal és shift gombok lenyomását figyeli, ellenőrzi, hogy a lépés lehetséges-e, ha igen, akkor meglépi (lépések fentebb leírva)
- `public class FlowersLeftPanel` ~ az aktuális virágok számát kiíró panel, ami folyamatosan frissül
- `public boolean wall(int, int)` ~ ellenőrzi, hogy egy koordináta a `tileTable`-ben fal típusú-e(`KeyListener` segédfüggvénye)
- `Menu4_TwoPlayer.java`: A kétszemélyes játékot megjelenítő menü
 - `private Boards_TwoPlayer b1` ~ egy `JPanel` a `JFrame`-en belül, ami tulajdonképpen maga a játéktábla(amin belül a játékosok tudnak mozogni és virágot ültetni, és felvenni)
 - `public Menu4_TwoPlayer()` ~ beállítja, hogy hogy nézzen ki a menü (felül kiírja a játékosok nevét, és hogy hány viráguk van jelenleg, alul pedig a játék tábla látható)
 - `public class P1KeyListener()` ~ a fel, le, jobb, bal és shift gombok lenyomását figyeli, ellenőrzi, hogy a lépés lehetséges-e, ha igen, akkor meglépi (lépések fentebb leírva)
 - `public class P2KeyListener()` ~ a w, s, d, a és space gombok lenyomását figyeli, ellenőrzi, hogy a lépés lehetséges-e, ha igen, akkor meglépi (lépések fentebb leírva)
 - `public class FlowersLeftPanel` ~ az aktuális virágok számát kiíró panel, ami folyamatosan frissül
 - `public boolean wall(int, int)` ~ ellenőrzi, hogy egy koordináta a `tileTable`-ben fal típusú-e(`KeyListener` segédfüggvénye)
- `Menu5_OnePlayer.java`: A játék végeredményét kiíró menü(egyjátékos mód esetén)
 - `private JButton button` ~ Egy gomb, amit a `NextButtonListener` hallgat
 - `public Menu5_OnePlayer()` ~ beállítja a menü kinézetét (felül kiírja, hogy vége a játéknak, alatta pedig, hogy hány pontot szerzett a játékos, és alatta egy gomb)
 - `public class NextButtonListener()` ~ meghív egy példányt a `Menu6` class-ból és bezárja ezt
- `Menu5_TwoPlayer.java`: A játék végeredményét kiíró menü(kettő játékos mód esetén)
 - `private JButton button` ~ Egy gomb, amit a `NextButtonListener` hallgat
 - `public Menu5_TwoPlayer()` ~ beállítja a menü kinézetét (felül kiírja, hogy ki nyert, alatta pedig, hogy hány pontot szereztek a játékosok, és alatta egy gomb)
 - `public class NextButtonListener()` ~ meghív egy példányt a `Menu6` class-ból és bezárja ezt
- `Menu6.java`: Az utolsó menü, ami megnyílik, megkérdezi, hogy a score-t le szeretné-e menteni a játékos a ranglistába.
 - `private JButton button1` ~ a `SaveButtonActionListener` hallgatja le ezt a gombot

- `private JButton button2` ~ a `NoSaveButtonActionListener` hallgatja le ezt a gombot
- `public Menu6()` ~ a menü kinézetét állítja be (felül szöveg, alul két gomb egymás mellett)
- `public class NoSaveButtonListener()` ~ A gomb megnyomására leállítja a programot, hiszen vége a játéknak.
- `public class saveButtonActionListener()` ~ A gomb megnyomására lementi a játszmát egy ranglistába, ami egy JSON fájl .txt kiterjesztéssel. Ez megtalálható a program mappájában, és emberi szemmel is elolvasható
- `public Game winnerGet()` ~ készít egy játszmát (Game class egy példányát), hogy a nyertes rekordját le tudja menteni a leaderboard-ba, ha azt a játékos szerette volna.
- package boards:
 - `Tile.java`: Egy class a képeknek a játéktáblán való megjelenítéséhez, tulajdonképpen, csak ellenőrzi, hogy a megkapott kép neve valóban létezik-e , és azt elmenti egy privát változójaként, és ezt le is lehet kérni egy getter-rel
 - `private Image image` ~ a kép, amit tárol
 - van gettere (Image) (`getImage()`)
 - `public Tile(String)` ~ egy string paraméterként megadott fájlnevről ellenőrzi a létezését magának a fájlnek
 - `DrawPlayers.java`: kirajzolja a játékosokat, és beállítja a koordinátáikat
 - `public DrawPlayers(Player, int, int)` ~ A paraméterként megadott játékosnak állítja be a parameterként megadott sorra X és Y koordinátákat
 - `public void drawPlayer(Player, Graphics)` ~ ténylegesen megjeleníti a maraméterként megadott játékost a saját koordinátáin
 - `Boards.java`: Egy absztrakt osztály, ami a `JPanel` leszármaztatottja. Ez a megjelenítendő játék tábláknak az alapja.
 - `protected long time` ~ a pontok számításához szükséges a játék kezdetekori időt tárolja egy long-ként
 - van setter-e (void) (`set_time`)
 - `protected Tile wall` ~ a fal-nak a képét tárolja
 - `protected Tile flower1` ~ az 1-es játékos virágának képét tárolja
 - `public static cellSize` ~ a táblán egy négyzet mérete
 - van getter-e (int) (`getCellSize()`)
 - `protected int boardNumber` ~ a játéktáblának a száma, eddig két tábla van mind egy és kétjátékos módban
 - `protected String board` ~ egy egysoros string, ami eltárolja, hogy a `tileTable`-ben mi hol van (főleg falak)
 - `protected String filename` ~ a file neve, amiből beolvassa a board-ot, valamint az answers-t is
 - `protected ArrayList<String> answers` ~ ebben vannak azok a koordináták eltárolva, ahova lehet virágokat ültetni
 - `public Board()` ~ beállítja üresre az answer-t, valamint legenerál egy random számot
 - `protected void paintComponent(Graphics)` ~ meghívja a `paintComponent`-jét a `JPanel`nek

- `public int randomGenerator()` ~ generál egy random int-et most 1 és 2 között, hiszen egyes és kettes pályák vannak a több és egyjátékosban is
- `public void readOneLineFromText()` ~ beolvassa a még egy sorként tárolt `tileTable`-t a file-ból
- `public void answerSheetReading()` ~ beolvassa a megoldásait ugyanannak a pályának
- `Boards_OnePlayer.java`: Egy egyjátékos pályát hív meg, a `Boards` lezsármazottja.
 - `private long timespent` ~ ide mentődik a játékban eltöltött összes időnek az értéke `long`-ban
 - `public static String[][] tileTable` ~ ebben tárolódik, hogy a pálya melyik cell-jében mi van (pl.: fal, virág, semmi), a beállításnál, lekérésnél egy egy mezőjét lehet állítani a mátrixnak
 - van setter-e (`void`), és getter-e (`String`) is (`setTileTable(int, int, String)`, `getTileTable(int, int)`)
 - `public Boards_OnePlayer()` ~ beolvassa a file-ból a board-ot, beolvassa a megoldásokat, legenerálja a tábla keretét (minden legszéle fal), és legenerálja a tábla belsejét is
 - `protected void paintComponent(Graphics)` ~ megrajzolja a hátteret, megrajzolja a vonalakat, a játékosokat, a képeket (fal, virág), és megnézi, hogy vége van-e a játéknak
 - `public void basicBoardLookGenerator()` ~ a tábla szélét beállítja, hogy végig fal legyen
 - `public void boardLookGenerator()` ~ a tábla közepénél beállítja, hogy hol vannak falak (közepe = minden, ami nem a kerete a pályának)
 - `public void drawLines(Graphics g)` ~ megrajzolja a rácsozást, hogy jól láthatóak legyenek, hogy melyik útnak hány fal szomszédja van
 - `public void drawPlayer(Graphics g)` ~ megrajzolja a `player1` karaktert a pályán
 - `public void drawImages(Graphics g)` ~ megrajzolja a falakat, meg a virágokat
 - `public void endOfGameCheck()` ~ megnézi, hogy a `player1`-nak van-e még virágja, és ha nincs akkor befejezi a játékot, kiszámolja a `timespent`-et és beállítja és megnyitja az utolsó előtti menüt, kiszámolja a játékos pontját is
 - `public void check()` ~ ebben a segésfüggvényben van leírva a pontszámítás pontos menete (végignézi, hogy az `answers` koordinátáin van-e virág, stb...)
- `Boards_TwoPlayer.java`
 - `private long timespent` ~ ide mentődik a játékban eltöltött összes időnek az értéke `long`-ban
 - `public static String[][] tileTable` ~ ebben tárolódik, hogy a pálya melyik cell-jében mi van (pl.: fal, virág, semmi), a beállításnál, lekérésnél egy egy mezőjét lehet állítani a mátrixnak
 - van setter-e (`void`), és getter-e (`String`) is (`setTileTable(int, int, String)`, `getTileTable(int, int)`)
 - `private Tile flower2` ~ a `player2` által elültethető virágok képét tárolja `tile`-ként

- `public Boards_TwoPlayer()` ~ beolvassa a file-ból a board-ot, beolvassa a megoldásokat, legenerálja a tábla keretét (minden legszéle fal), és legenerálja a tábla belsejét is
- `protected void paintComponent(Graphics)` ~ megrajzolja a hátteret, megrajzolja a vonalakat, a játékosokat, a képeket (fal, virág), és megnézi, hogy vége van-e a játéknak
- `public void basicBoardLookGenerator()` ~ a tábla szélét beállítja, hogy végig fal legyen
- `public void boardLookGenerator()` ~ a tábla közepénél beállítja, hogy hol vannak falak (közepe = minden, ami nem a kerete a pályának)
- `public void drawLines(Graphics g)` ~ megrajzolja a rácsozást, hogy jól láthatóak legyenek, hogy melyik útnak hány fal szomszédja van
- `public void drawPlayers(Graphics g)` ~ megrajzolja a player1 karaktert, és a player2 karaktert is a pályán
- `public void drawImages(Graphics g)` ~ megrajzolja a falakat, meg a virágokat
- `public void endOfGameCheck()` ~ megnézi, hogy a player1-nak vagy a player2-nak van-e még virágja, és ha egyiküknek nincs akkor befejezi a játékot, kiszámolja a timespent-et és beállítja és megnyitja az utolsó előtti menüt, kiszámolja a játékosok pontját is
- `public void check()` ~ ebben a segítségfüggvényben van leírva a pontszámítás pontos menete (végignézi, hogy az answers koordinátáin van-e virág, stb...)